



TECHNO INDIA UNIVERSITY
WEST BENGAL

Stock Prediction and Management

8th Semester Project Report

submitted to

Techno India University, West Bengal

In partial fulfilment of the requirements
for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering

by

Mayukh Karmakar (Roll No. 221001001204)

Debjyoti Sasmal Hui (Roll No. 221001001199)

Akshat Raj Verma (Roll No. 221001001198)

Suvojit Dutta (Roll No. 221001001206)

Tathagata Hui (Roll No. 221001001201)

Under the guidance of

Prof. Jayanta Chaudhuri

Department of Computer Science & Engineering

TECHNO INDIA UNIVERSITY, WEST BENGAL

June 2026

© 2026, Techno India University. All rights reserved.

DEDICATED TO

Firstly, to The Almighty God, whose blessings of grace, wisdom, and guidance have been the source of strength for us throughout this process.

To our esteemed Project Supervisor, Prof. Jayanta Chaudhuri, for his guidance and valuable advice, and to our Head of Department, Dr. Kalpita Dutta for her vision and leadership which created an enabling academic atmosphere for us.

To all the faculty members of the Department of Computer Science and Engineering, Techno India University, who have through their guidance in eight semesters laid down the strong foundation of knowledge for this work.

DECLARATION OF AUTHORSHIP

We hereby declare that the project report entitled **Stock Price Prediction and Management** is an authentic record of our own work carried out at the **Department of Computer Science and Engineering, Techno India University, West Bengal**, during the 8th semester of the academic year 2025-2026 under the supervision of **Jayanta Chaudhuri**, Assistant Professor.

We further declare that the matter embodied in this project report has not been submitted by us for the award of any other degree or diploma of this or any other Institute/University. All the information have been obtained and presented in accordance with academic rules and ethical conduct. We also declare that, as required by these rules and conduct, we have fully cited and referenced all materials and results that are not original to this work.

Place: EM-4/1, Sector V, Bidhannagar, Kolkata, West Bengal, 700091

Date: 03-06-2026

Signatures:

Student 1 Name: Mayukh Karmakar, Roll No: 221001001204, Signature:

Student 2 Name: Tathagata Hui, Roll No: 221001001201, Signature:

Student 3 Name: Suvojit Dutta, Roll No: 221001001206, Signature:

Student 4 Name: Akshat Raj Verma, Roll No: 221001001198, Signature:

Student 5 Name: Debjyoti Sasmal, Roll No: 221001001199, Signature:

CERTIFICATE OF RECOMMENDATION

This is to certify that the work embodied in this thesis entitled **Stock Price Prediction and Management** has been satisfactorily completed by **Mayukh Karmakar** (Roll No. 221001001204), **Tathagata Hui** (Roll No. 221001001201), **Suvojit Dutta** (Roll No. 221001001206), **Akshat Raj Verma** (Roll No. 221001001198), and **Debjyoti Sasmal** (Roll No. 221001001199). It is a bonafide piece of work carried out under my supervision and guidance at Techno India University, Kolkata, for partial fulfilment of the requirements for the awarding of the **Bachelor of Technology in Computer Science & Engineering** degree of the Department of Computer Science and Engineering, Techno India University, during the academic year 2025 – 2026.

PROF. JAYANTA CHAUDHURI,

Assistant Professor, Department of Computer Science and Engineering, Techno India University, Kolkata, West Bengal, India.

(Supervisor)

Forwarded By:

DR. KALPITA DUTTA,

Assistant Professor & HOD, Department of Computer Science and Engineering, Techno India University, Kolkata, West Bengal, India.

ACKNOWLEDGMENT

We would like to extend our deepest gratitude to Techno India University and the Faculty of Engineering, particularly the Department of Computer Science and Engineering, for providing us with the invaluable opportunity, infrastructure, and academic resources necessary to complete this project as part of our Bachelor of Technology curriculum. Their commitment to fostering an environment of innovation and practical learning has been instrumental to our progress.

We would also like to express our sincere appreciation and gratitude to our project supervisor, Prof. Jayanta Chaudhuri. His constant guidance, insightful feedback, and unwavering support throughout every stage of this project were pivotal in shaping our work. His expertise and encouragement not only helped us overcome technical challenges but also provided us with a deeper understanding of the subject matter.

This project was a collaborative effort by the following team members: Mayukh Karmakar, Suvojit Dutta, Tathagata Hui, Debjyoti Sasmal and Akshat Raj Verma.

We take pride in our collective contribution and are grateful for the synergy and dedication each member brought to the team, which allowed us to successfully achieve our project goals. Finally, we would like to thank our families and friends for their constant encouragement and patience, which served as a great source of motivation throughout the duration of this endeavour.

ABSTRACT

Retail investors and students often lack access to a single tool that connects equity forecasting, market context, and hands-on trading practice in one place. This project addresses that need by building a web-based Stock Price Prediction and Management platform that brings together machine-learning forecasting, news-driven sentiment analysis, and a risk-free simulated brokerage — all within a unified Flask and SQLite application. At the core of the system is a dual-mode Linear Regression engine that projects closing prices seven days ahead. Users choose between a Close Price model, which draws on historical pricing patterns, and a volume-Based model, which uses trading activity as its primary signal. Each mode displays a catalyst badge alongside its RMSE score so users can evaluate forecast quality briefly. Alongside the price projections, a multi-source news aggregator scores recent financial headlines as positive, negative, or neutral, giving users qualitative market context to complement the quantitative forecast. The paper-trading module supports account registration, a virtual cash wallet, buy and sell order execution with adjustable commission rates, dividend logging, live portfolio valuation, and a searchable transaction history. Every completed trade automatically generates a downloadable plain-text invoice, while an in-app notification layer keeps users informed of order confirmations and account activity. Portfolio Snapshot reports can be produced on demand for any of five-time windows — 7, 30, 90, or 365 days, or the full account history — supporting structured performance review. System reliability is maintained through size-based log rotation, HTTP middleware that records per-request latency and error counts, and a /health endpoint for external monitoring. Administrators access a dedicated dashboard exposing live traffic metrics, recent log entries, and a Dataset Management interface for uploading, inspecting, and removing the CSV files used to train the forecasting model. Taken together, these capabilities close the gap that typically exists between standalone prediction tools and a fully functional portfolio management environment, making the platform a practical resource for learning and experimentation in equity markets.

CONTENTS

1	INTRODUCTION	10
	1.1 Background	10
	1.2 Problem Definition	10
	1.3 Project Overview	11
	1.4 Scope	11
	1.5 Technology Stack	12
2	LITERATURE REVIEW	13
	2.1 Stock Market Forecasting	13
	2.2 Linear Regression for Stock Price Prediction	13
	2.3 Multiple Linear Regression and Feature Selection	13
	2.4 Gaps Identified	14
3	OBJECTIVES	15
	3.1 Project Objectives	15
	3.2 Scope Summary	15
4	METHODOLOGY	16
	4.1 System Requirements	16
	4.2 Data Collection and Preprocessing	16
	4.3 Linear Regression Forecasting Method	17

	4.4 Sentiment Analysis Method	17
	4.5 System Architecture and Design	18
	4.6 Implementation Overview	20
5	RESULTS AND DISCUSSIONS	21
	5.1 Evaluation Metrics	21
	5.2 Output Screens	21
	5.3 Discussion	25
6	FUTURE WORK	26
	6.1 Conclusion	26
	6.2 Future Enhancements	26
7	BIBLIOGRAPHY	27
8	APPENDICES	28
	8.1 Appendix A	28
	8.2 Appendix B	29
	8.3 Appendix C	30

LIST OF FIGURES

Figure-1: Use Case Diagram	18
Figure-2: Level 0 Data Flow Diagram (DFD Level 0)	18
Figure-3: Portfolio Management (DFD Level 1)	19
Figure-4: Entity Relationship Diagram	19
Figure-5: Project Gantt Chart	20
Figure-6: Home / Prediction Studio page	21
Figure-7: LR model test graph	21
Figure-8: Sentiment Analysis	22
Figure-9: Stock Trading	22
Figure-10: Notifications	23
Figure-11: Admin Dashboard	23
Figure-12: Data Logging	24
Figure-13: Dataset Management Interface	24

LIST OF TABLES

1.1	Technology stack	12
1.2	Users Table	29
1.3	Trades Table	29

CHAPTER 1: INTRODUCTION

Retail investors and students often juggle separate tools for price forecasting, news analysis, and portfolio tracking. This project combines all three in a single Flask application. The forecasting engine provides two modes: a Close-price model trained on historical data, and a Volume-based model that incorporates trading activity.

1.1 Background

Pattern-based price analysis and sentiment analysis each tell only part of the story — historical trends show where a stock has been, while news sentiment captures the market's current mood. This project merges into a single decision-support layer.

1.2 Problem Definition

In the modern age, equity analysis often tends to be narrow minded applications - machine learning models rarely come with trading interfaces and it's not rare to find paper-trading applications missing predictive analytics. This project manages to resolve this by providing a playground that not only handles the current valid stock data but comes prepackaged with short-term price projection, performance metrics, news-based sentiment scoring, and a secure stock trading and brokerage system.

1.3 Project Overview

The Stock Price Prediction and Management platform is a self-contained web application that delivers:

- Short-term equity price forecasting using a Linear Regression model
- Automated sentiment scoring drawn from current financial news sources
- A paper-trading module with full portfolio oversight and an administrative control panel

1.4 Scope

1.4.1 In Scope

- End-to-end Flask web application
- Role-based access control (standard user and administrator tiers)
- Market data ingestion via yfinance (primary) and Alpha Vantage (fallback)
- Linear Regression price forecasting over a 7-day horizon, with Closing Price and Volume-Driven modes
- Multi-source news sentiment aggregation with fallback logic
- Portfolio management: holdings, buy/sell execution, dividend processing, virtual cash, and commission handling
- In-app alert system for trade events, billing statements, and analytical reports, with per-user preference controls
- Portfolio Snapshot reports across selectable periods (7, 30, 90, 365 days, or all-time)
- Automatic invoice generation (plain-text) for every Purchase, Sale, and Dividend event
- Admin dashboard covering brokerage configuration, transaction review, user management, company records, aggregate statistics, live HTTP metrics, system event logs, and dataset governance
- Operational monitoring via a /health endpoint tracking response times, error rates, and uptime
- Rotating structured logging to logs/app.log
- New database entities: UserNotification, NotificationPreference, ReportSnapshot, Invoice

1.4.2 Out of Scope

- Live brokerage connectivity or real-money order execution
- Advanced ML Ops capabilities such as automated model retraining, version registries, or experiment tracking
- Enterprise-scale infrastructure including distributed caching, asynchronous task queues, or multi-tenant deployments

1.5 Technology Stack

TABLE 1.1 Technology Stack

Layer	Core Technologies	Libraries
Backend	Python, Flask	Flask-SQLAlchemy, requests, Aiohttp
Database	SQLite	SQLAlchemy(ORM)
ML & Analytics	Python	scikit-learn, pandas, numpy
Sentiment/ NLP	Python	NLTK Vader, TextBlob, BeautifulSoup, newspaper3k, tenacity
Frontend	HTML/ CSS, JS	Bootstrap, D3.js

CHAPTER 2: LITERATURE REVIEW

2.1 Stock Market Forecasting

The strategically proven choice for selecting Linear Regression over some of the other complex deep learning models is because Linear Regression is computationally cheap for short-term predictions and fast to run as documented by Lavanya and Gnanasskaran [1]. Lavanya and Gnanasskaran [1] also noted in their research that simple linear models were able to perform reliably, while keeping deployment overhead low.

2.2 Linear Regression for Stock Price Prediction

These studies suggest linear regression remains a practical choice for stock price prediction, especially when paired with well-chosen inputs. [2] Sangeetha and Alfia tested a linear regression method and found that it gives a good result without needing a lot of computing power, which is very useful for websites and apps where speed is important. We also found out that if you choose the right input data carefully, linear regression gives reliable predictions. Another big advantage of this method is that it is easy to understand as you can clearly see which factors are affecting the predicted price.

2.3 Multiple Linear Regression and Feature Selection

Many studies have shown that linear regression works well for short-term stock price prediction. [2] Sangeetha and Alfia found that it delivers reliable results without heavy computation, making it practical for web applications where speed matters. Their work also highlights that careful feature selection significantly improves accuracy. A further advantage is the model's interpretability — it's straightforward to see which factors drive the prediction.

2.4 Gaps Identified

Most of these studies use linear regression for price prediction, but few explore how predictions can feed into a portfolio management workflow. This project bridges that by combining regression-based prediction with engineered features, scraped news headlines for market context, and a full paper-trading system — complete with broker commissions, user trading, and an admin dashboard.

CHAPTER 3: OBJECTIVES

3.1 Project Objectives

- Price Forecasting: Generate 7-day ahead price projections using a Linear Regression model
- Sentiment Analysis: Retrieve and score recent news coverage to produce positive/negative/neutral polarity readings
- Decision Support: Combine directional price forecasts with sentiment scores into actionable trading signals
- Portfolio Simulation: Provide virtual cash management, position tracking, buy/sell workflows, brokerage fees, dividend processing, and a full transaction history
- Administration: Supply an admin monitoring panel and brokerage configuration controls

3.2 Scope Summary

This platform is made strictly for educational purposes; it is mainly used to track market sentiment and portfolio simulation.

CHAPTER 4: METHODOLOGY

4.1 System Requirements

4.1.1 Hardware Requirements (Minimum)

- Processor: Dual-core CPU or better
- RAM: 4 GB (8 GB recommended)
- Storage: 1 GB free space
- Internet: Required (market data + news sentiment)

4.1.2 Software Requirements

- Operating System: Windows / Linux / macOS
- Python: 3.7+
- Libraries: as listed in requirements.txt

4.2 Data Collection and Preprocessing

4.2.1 Market Data Acquisition

When someone types in a stock name, the system first checks if it already has a saved {TICKER}.csv file on the computer. If the file is already there and up to date, the app just uses it to save time. If the file is missing or too old, the app uses yfinance to download about two years of old price data from the internet. If yfinance has an issue or fails to load, the system automatically switches to Alpha Vantage as a backup to get the data.

4.2.2 Data Cleaning

- Rows containing missing values are removed using dropna
- The Close price column serves as the primary predictive signal

4.3 Linear Regression Forecasting Method

The forecasting model is based on the previously done research [1, 2, 3], which works as follows:

- A forecast window of a week is selected to get an estimate of price movements.
- Now, based on this target labels are created by shifting Close columns by n number of days.
- The pattern and price of the stock are collected from the historical dataset.
- Now all properties are normalised by using a StandardScaler that is consistent [2].
- Then an 80/20 train-test split is applied before proceeding with the linear regression model.
- Therefore, the trained model forecast closing prices for the next 7 trading days.
- The accuracy of the model is given as RMSE based on the 20% test set.
- For observed systematic market tendencies a 4% calibration factor is implemented.

Hence, this design achieves a workable balance between the predictive quality and response speed that makes the model suitable for web applications [1, 4].

4.4 Sentiment Analysis Method

Multiple articles and headlines relating to the stock are retrieved from various news sources , on the basis of this a sentiment score is computed for each piece of content and then aggregated .The final output results in the overall polarity (+ve , -ve or neutral) along with a count of articles used in each category.

4.5 System Architecture and Design

4.5.1 Architectural Overview

- Presentation Layer: HTML templates and dashboards
- Application Layer: Flask routes and business logic
- Data Layer: SQLite database via SQLAlchemy

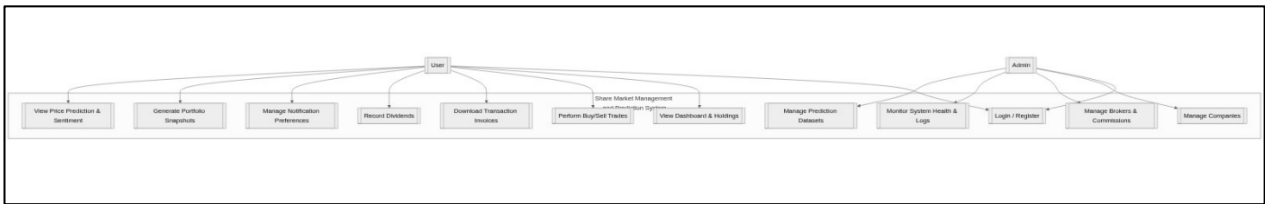


Figure 1: 4.5.2 Use Case Diagram

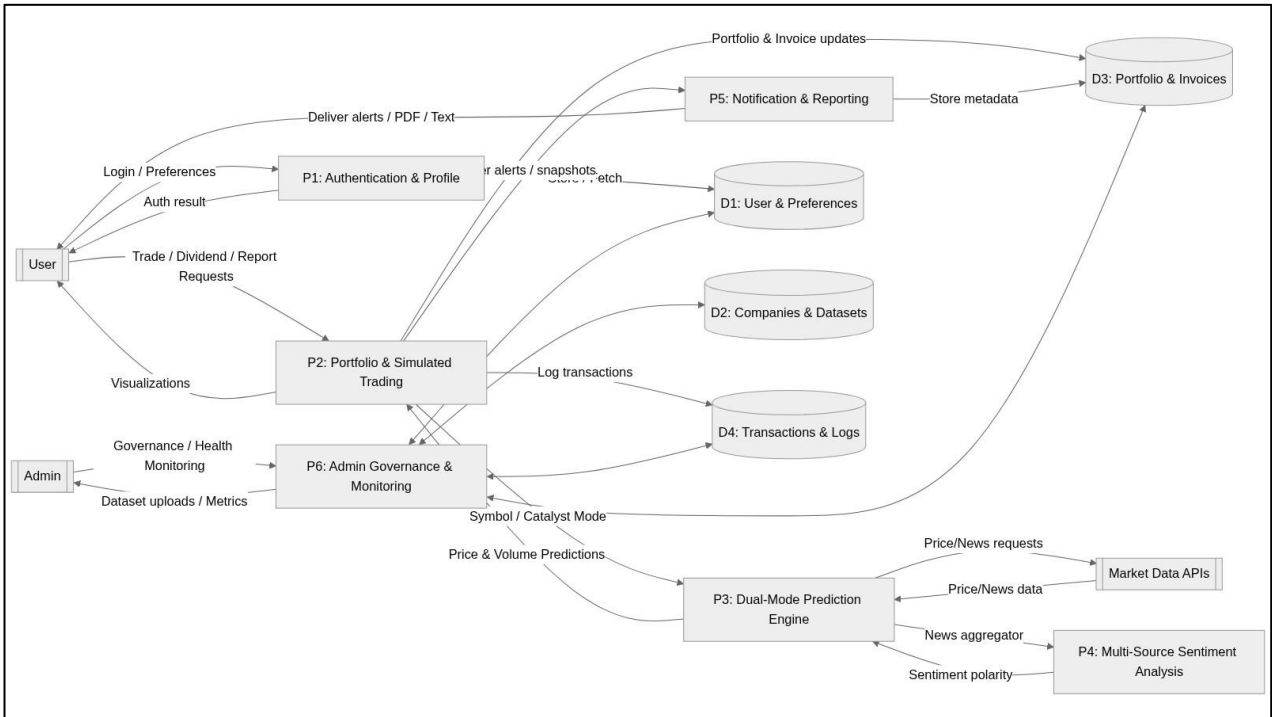


Figure 2: 4.5.3 Data Flow Diagram (Level 0)

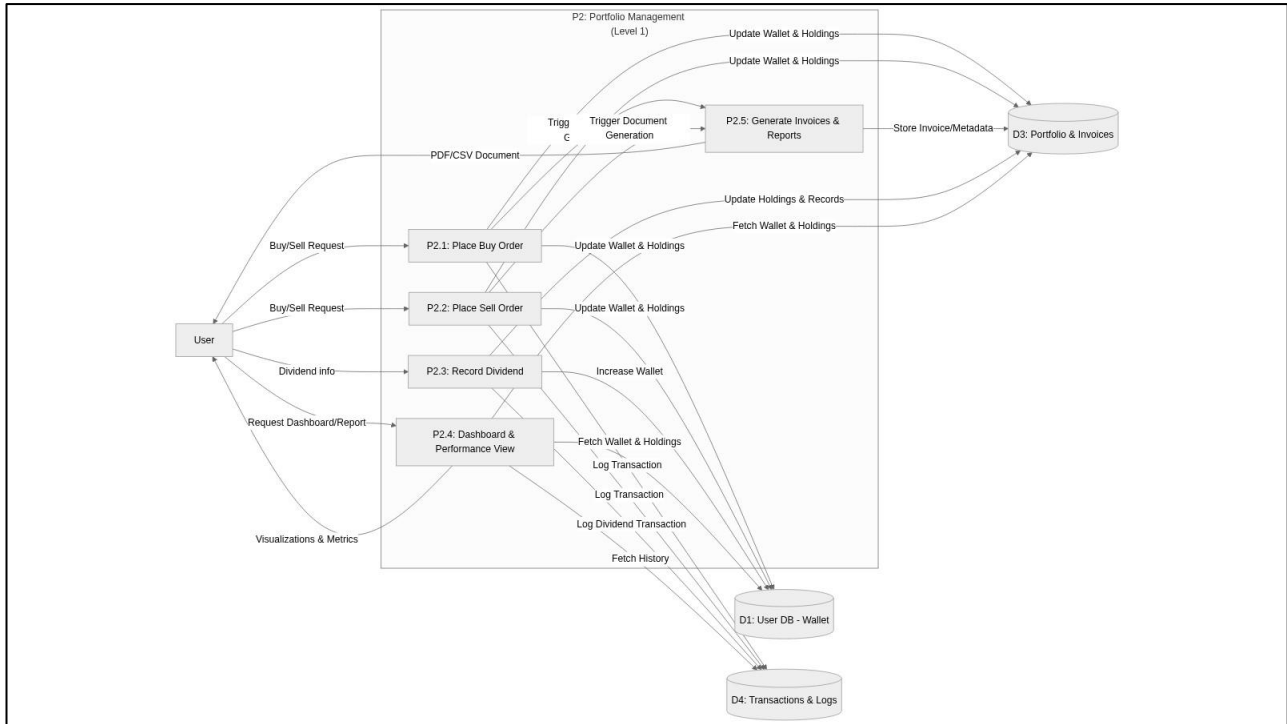


Figure 3: 4.5.4 Portfolio DFD (Level 1)

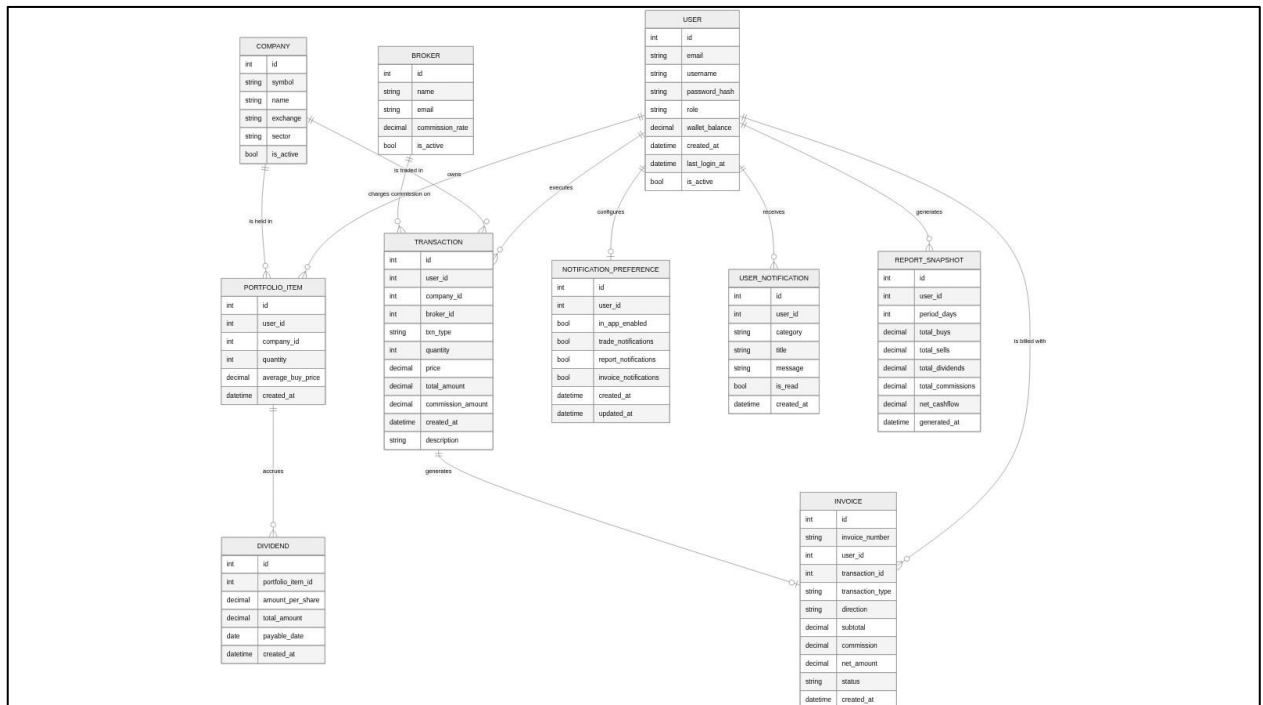


Figure 4: 4.5.5 Database Design (ER Diagram)

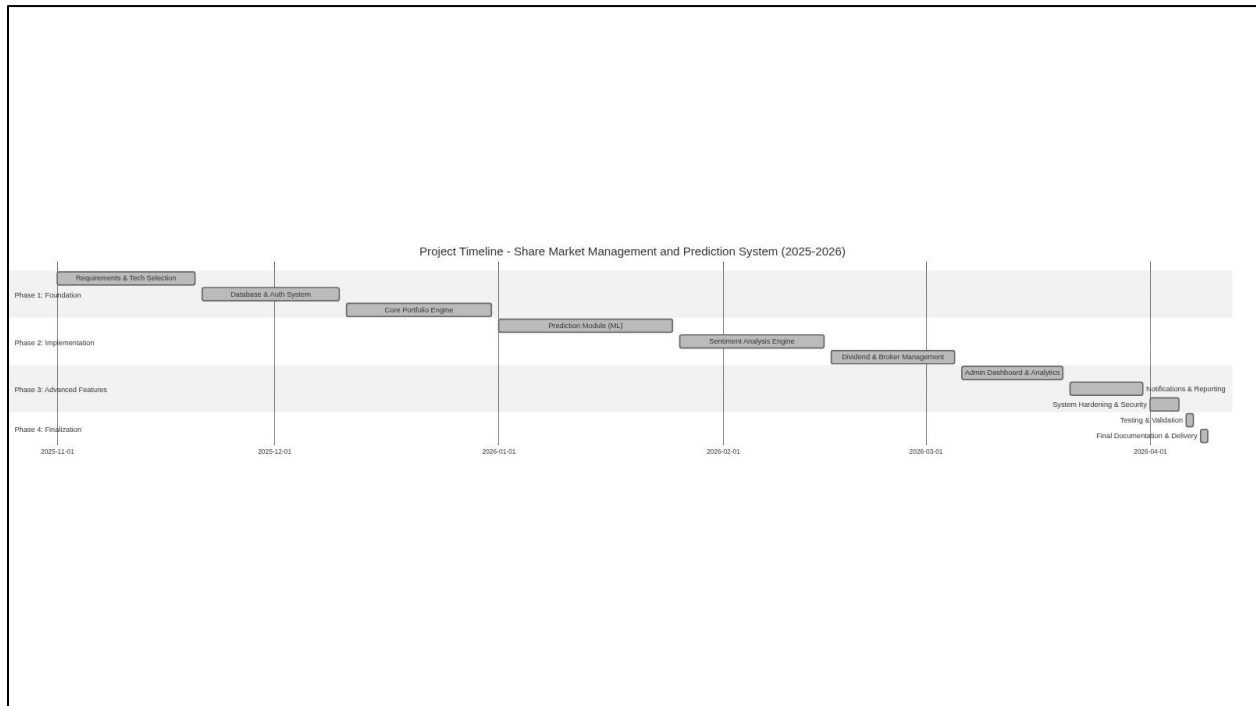


Figure 5: 4.5.6 Project Plan (Gantt Chart)

4.6 Implementation Overview

- **main.py:** This is the main file of the application. It handles all the Flask routes, defines the database models, and manages how the machine learning predictions are run.
- **news_sentiment.py:** A separate module that is used specifically for running the sentiment analysis.
- **The 'templates' folder:** A folder containing the Jinja2 HTML templates for the user dashboard, the prediction results page, and the admin panel.
- **static:** Contains all frontend assets: CSS, client-side JavaScript, and D3.js chart scripts.
- **Monitoring:** Tracks request metrics, exposes a health-check endpoint, and writes rotating log files for debugging.
- **Notifications & Settings:** Users can view alerts and update preferences in-app, backed by notification, report, and invoice tables

CHAPTER 5: RESULTS AND DISCUSSIONS

5.1 Evaluation Metrics

Root Mean Squared Error (RMSE) is computed for the Linear Regression model and is displayed on the results page. This gives users an overview of the forecast accuracy against the test data.

5.2 Output Screens

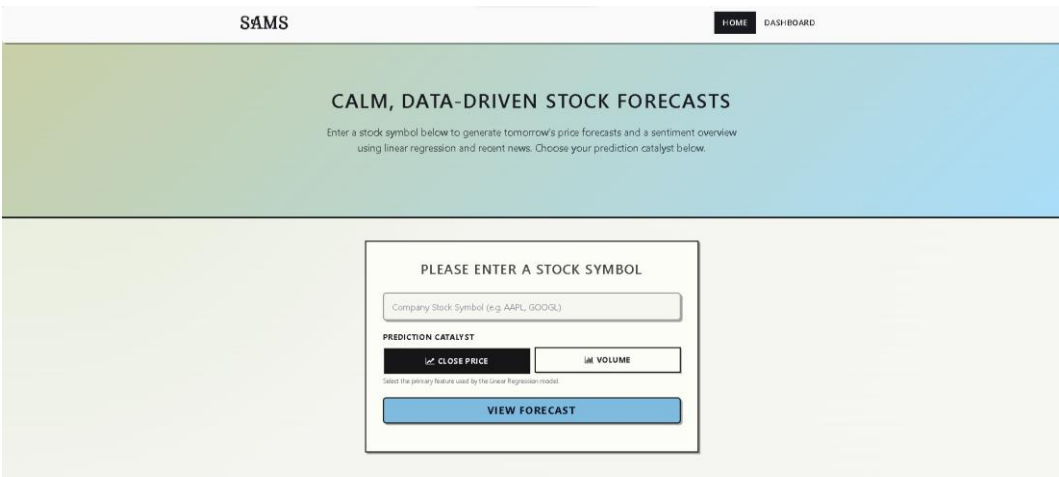


Figure 6: Home / Prediction Studio page

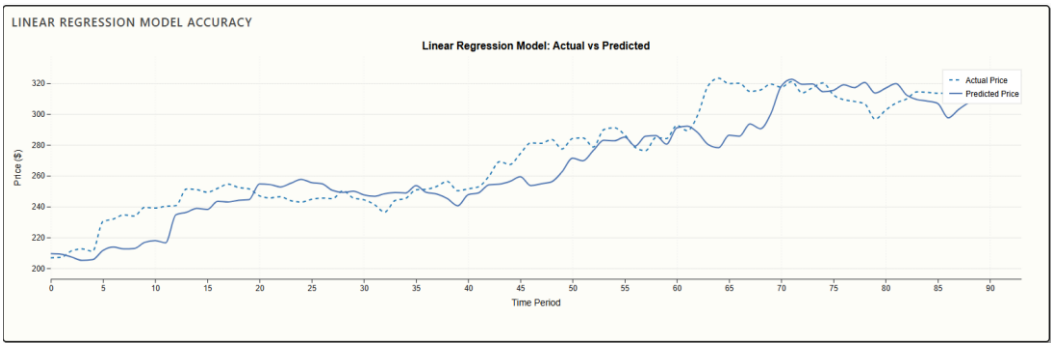


Figure 7: LR model test graph

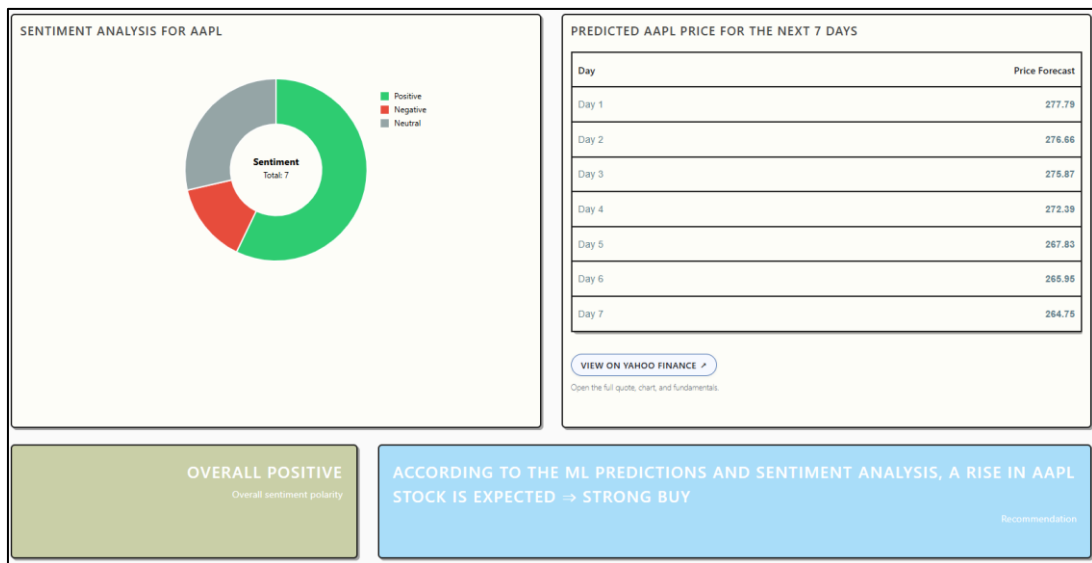


Figure 8: Sentiment Analysis

9656736.64

WALLET BALANCE

607301.07

CURRENT PORTFOLIO VALUE

ADD FUNDS (SIMULATION)

Amount [TOP UP](#)

RECORD DIVIDEND

Symbol Amount per share [RECORD](#)

HOLDINGS

Symbol	Quantity	Current Price	Actions		
AAPL	834	260.46 +1.09 (+0.42%)	Qty	BUY	Qty SELL VIEW PREDICTION
TSLA	471	436.86 -8.15 (-1.83%)	Qty	BUY	Qty SELL VIEW PREDICTION
NVDA	1009	182.67 -2.19 (-1.18%)	Qty	BUY	Qty SELL VIEW PREDICTION

NEW TRADE

Symbol Quantity [BUY](#) [SELL](#)

RECENT TRANSACTIONS (BILLING)

Figure 9: Stock Trading

NOTIFICATION SETTINGS

☒ Enable in-app notifications
 ☒ Trade updates
 ☒ Report generation updates
 ☒ Invoice updates

SAVE SETTINGS

REPORTING SYSTEM

Last 30 Days

GENERATE REPORT

No reports generated yet.

RECENT NOTIFICATIONS

Time	Title	Message	Status
No notifications yet.			

ENHANCED INVOICES

Time	Invoice #	Type	Direction	Net Amount	Action
No invoices generated yet.					

REPORT HISTORY

Generated At	Period (Days)	Buys	Sells	Dividends	Commissions	Net Cashflow
No reports in history yet.						

Figure 10: Notifications

1
TOTAL USERS

0
BROKERS

0
TRANSACTIONS

0
COMPANIES

0.00
TOTAL COMMISSION

0
TOTAL VOLUME (SHARES)

0
INVOICES

0
REPORT SNAPSHOTS

0
NOTIFICATIONS

593
UPTIME (SEC)

158
TOTAL REQUESTS

174.89
AVG LATENCY (MS)

0
SERVER ERRORS

REGISTER BROKER

Broker name

Email

Commission %

ADD BROKER

BROKERS

Name	Email	Commission %	Active
No brokers registered.			

Figure 11: Admin Dashboard

COMPANY PROFILES				
Symbol	Name	Exchange	Sector	Active
No companies configured yet. They will be created automatically when users trade.				

RECENT TRANSACTIONS							
Time	User	Type	Symbol	Qty	Price	Total	Description
No transactions yet.							

MONITORING LOGS (LATEST)	
	Log Entry
	2026-04-09 10:57:16,895 INFO HTTP GET /static/animate.min.css -> 200 (0.55 ms)
	2026-04-09 10:57:16,896 INFO HTTP GET /static/neubrutalist.css -> 200 (1.38 ms)
	2026-04-09 10:57:16,898 INFO HTTP GET /static/simple-line-icons.css -> 200 (1.50 ms)
	2026-04-09 10:57:16,898 INFO HTTP GET /static/weather-icons.min.css -> 404 (0.52 ms)
	2026-04-09 10:57:16,901 INFO HTTP GET /static/linea.css -> 200 (1.39 ms)
	2026-04-09 10:57:17,113 INFO HTTP GET /icons/cryptocoins/css/cryptocoins.css -> 404 (0.11 ms)
	2026-04-09 10:57:17,211 INFO HTTP GET /icons/themify-icons/themify-icons.css -> 404 (0.12 ms)

Figure 12: Data Logging

UPLOAD DATASET

CSV FILE

Click or drag a .csv file here

Upload a stock CSV (e.g. AAPL.csv) produced by yfinance or Alpha Vantage. Overwrites existing file with same name.

UPLOAD DATASET

DATASET FILES13CSVs in app root

Filename	Size	Last Modified	Action
AAPL.csv	46.5 KB	2026-04-09 10:51	DEL
abnb.csv	44.4 KB	2026-04-08 19:37	DEL
admin@example.com,admin123.csv	0.0 KB	2026-04-08 19:37	DEL
azn.csv	44.8 KB	2026-04-08 19:37	DEL
GOOG.csv	46.6 KB	2026-04-08 19:37	DEL
GOOGL.csv	46.5 KB	2026-04-09 10:44	DEL
HDFCBANK.csv	0.0 KB	2026-04-08 19:37	DEL
hindaico.csv	0.0 KB	2026-04-08 19:37	DEL
MSFT.csv	46.0 KB	2026-04-08 19:37	DEL
tcs.csv	0.0 KB	2026-04-08 19:37	DEL
tsla.csv	44.5 KB	2026-04-08 19:37	DEL
wipro.csv	0.0 KB	2026-04-08 19:37	DEL
Yahoo-Finance-Ticker-Symbols.csv	4787.0 KB	2026-04-08 19:37	DEL

Filter by filename...

Figure 13: Dataset Management Interface

5.3 Discussion

- The system provides a unified view combining technical forecasting and sentiment context, addressing the integration gap identified in the literature [1], [5].
- The Linear Regression model delivers fast response times suitable for web applications while maintaining interpretable predictions [2], [3].
- Feature scaling and proper train-test splitting ensure robust model performance as recommended by recent studies [2], [4].
- The 7-day forecast horizon provides actionable short-term predictions that align with practical trading strategies.
- The sentiment engine is designed with multiple fallback sources to remain operational even when primary data sources are unavailable.
- The combination of quantitative forecasting and qualitative sentiment analysis provides a more comprehensive decision-support framework than either approach alone.

CHAPTER 6: FUTURE WORK

6.1 Conclusion

This system delivers a linear regression-based price prediction [1], [2], [3] combined with news sentiments aggregated from multiple sources and a robust portfolio management system.

This system successfully proves that the heavy computation involved in deploying the website app remains steady all the while maintaining the effectiveness of the linear regression models short term prediction [4], [5].

This system serves as a means of preserving quantitative forecasting with qualitative sentiment analysis which proves this project has the potential of being a practical tool for learning and experimenting with stock market trading.

6.2 Future Enhancements

- Expanding the forecasting module to support additional models and richer feature sets.
- Strengthening validation through systematic testing (unit, integration, performance, and security testing).
- Full-scale Electron app deployment.

BIBLIOGRAPHY

- [1] M. Lavanya and P. Gnanasskaran, "Stock Exchange Price Prediction Using Linear Regression Model," in Proc. 5th Int. Conf. Inventive Res. Computer. Appl. (ICIRCA), Coimbatore, India, 2023, pp. 1-6. DOI: 10.1109/ICIRCA57980.2023.10220627.
- [2] J. M. Sangeetha and K. J. Alfia, "Financial stock market forecast using evaluated linear regression-based machine learning technique," Meas.: Sensors, vol. 31, no. Supplement C, pp. 100950, Feb. 2024. DOI: 10.1016/j.measen.2023.100950.
- [3] S. Li, "Stock Price Prediction Based on Linear Regression and Significance Analysis," in Proc. 2nd Int. Conf. Data Sci. Eng. (ICDSE), 2025, pp. 596-603. DOI: 10.5220/0013702600004670.
- [4] H. Lin, "Predicting Stock Returns Using Linear Regression," Frontiers Bus., Econ. Manage., vol. 18, no. 3, pp. 139-141, 2025. DOI: 10.54097/06ck4c50.
- [5] R. Hu, "Stock Price Prediction Based on Multiple Linear Regression Model," in Proc. Int. Conf. Finance, Trade Bus. Manage. (FTBM), vol. 264, 2023, pp. 440-447. DOI: 10.2991/978-94-6463-298-9_48.

APPENDICES

8.1 Appendix A: Database Data Dictionary

1. Linear Regression Forecasting Logic

```
1. import pandas as pd
2. from sklearn.linear_model import LinearRegression
3. from sklearn.preprocessing import StandardScaler
4. from sklearn.model_selection import train_test_split
5.
6. def train_forecast_model(data, mode='close'):
7.     # Prepare features and labels
8.     if mode == 'volume':
9.         features = data[['Close', 'Volume', 'High', 'Low']]
10.    else:
11.        features = data[['Close', 'Open', 'High', 'Low']]
12.
13.    # Shifting for 7-day forecast window
14.    labels = data['Close'].shift(-7).dropna()
15.    features = features.iloc[:-7]
16.
17.    # Scaling
18.    scaler = StandardScaler()
19.    X = scaler.fit_transform(features)
20.    y = labels.values
21.
22.    # Train/Test Split (80/20)
23.    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
24.
25.    model = LinearRegression()
26.    model.fit(X_train, y_train)
27.
28.    return model, scaler
29.
```

2. Sentiment Analysis Aggregator

```
1. from textblob import TextBlob
2. from nltk.sentiment.vader import SentimentIntensityAnalyzer
3.
4. def get_sentiment_score(headlines):
5.     sid = SentimentIntensityAnalyzer()
6.     scores = []
7.
8.     for text in headlines:
9.         vader_score = sid.polarity_scores(text)['compound']
10.        blob_score = TextBlob(text).sentiment.polarity
11.        # Weighted average of multiple engines
12.        combined = (vader_score * 0.7) + (blob_score * 0.3)
13.        scores.append(combined)
14.
15.    avg_score = sum(scores) / len(scores) if scores else 0
16.    return "Positive" if avg_score > 0.05 else "Negative" if avg_score < -0.05 else "Neutral"
17.
```

8.2 Appendix B: Database Data Dictionary

1. Entity: Users

Table 2.1 Users Table

Field (ID)	Type (INTEGER)	Description (Primary Key)
Username	String (50)	Unique username for login
Email	String (120)	User contact email
password_hash	String (128)	Hashed password (Bcrypt)
role	String (20)	User or Admin tier
Balance	Float	Virtual cash available for trading

2. Entity: Trades

Table 2.2 Trades Table

Field (ID)	Type (INTEGER)	Description (Primary Key)
User_id	Integer	Foreign Key to Users
ticker	String (10)	Stock symbol (e.g., AAPL)
trade_type	String (10)	Buy or Sell
quantity	Integer	Number of shares transacted
price	Float	Execution price per share
timestamp	DateTime	Date and time of the trade

3. New Operational Entities

- **UserNotification:** This entity is used to store in-app alerts to confirm a trade or when the system updates.
- **ReportSnapshot:** This entity is used to store JSON blobs of portfolio performance data helping in the comparison of historical data.
- **Invoice:** This entity is used to auto-generate plain-text invoices for all financial events.

8.3 Appendix C: Sample Artifacts and Reports

1. Sample Plain-Text Invoice

```
1. =====
2.      STOCK MANAGEMENT PLATFORM INVOICE
3. =====
4. Invoice ID: INV-2026-0409-001
5. Date: 2026-04-09 14:30:22
6. User: Hare Krishna Chattopadhyay
7. -----
8. TRANSACTION DETAILS:
9. Type: BUY ORDER
10. Ticker: TSLA
12. Quantity: 10 Shares
13. Execution Price: ₹175.40
14. -----
15. Subtotal: ₹1,754.00
16. Brokerage Fee (0.5%): ₹8.77
17. TOTAL DEDUCTION: ₹1,762.77
18. -----
19. Status: COMPLETED
20. Thank you for using our simulated brokerage.
```

2. Portfolio Performance Snapshot (in json)

```
1. {
2.   "snapshot_id": "SNAP-9921",
3.   "period": "30_DAYS",
4.   "start_balance": 10000.00,
5.   "end_balance": 10450.25,
6.   "net_return": "+4.50%",
7.   "top_performer": "NVDA",
8.   "total_trades": 12,
9.   "generated_at": "2026-04-10T09:00:00Z"
10. }
11.
```